



PPNJ MIRROR PROJECT

Author: Nasrul Khair

Abstract

This report serves to document and reflect on my current responsibilities and ongoing initiatives as a Business Analyst at my present company. It outlines the key projects, methodologies, and analytical approaches I employ, providing a snapshot of my professional contributions and how they align with broader business objectives. The purpose of this documentation is to demonstrate the practical application of business analysis skills in a real-world corporate environment.

Table of Contents

Business Requirements Document (BRD)	1
Data Creation	4
Data Transformation	9
Data Validation & Quality Assurance (QA)	14
Data Upload to GCS	15
Query Analysis	18
Documentation & Data Dictionary	28
Exploratory Data Analysis (EDA)	8
Power BI Dashboard	30
KPI Dashboard / Summary Page	35
Insight Log / Analytical Journal	41
What-If / Scenario Modelling	44
Hypothesis Testing	48

BUSINESS REQUIREMENTS

Project Title:

Optimizing Travel Package Performance, Customer Satisfaction Analysis & Data Collection enhancement.

Stakeholder:

1. Mrs I, General Manager
2. Sales Department
3. Operations Team

Background & Business Need:

The company offers a variety travel packages across Malaysia and Southeast Asia. Management wants deeper insights into:

- Customers Behavior insight.
- Customer Feedback information.
- Sales Insight for the current year.
- Business Problem solutions
- Understanding the impact of operation activity towards customers satisfaction
- Modelling if certain case scenarios happen.

Objectives:

- Identify top-selling and high-satisfaction packages.
- Understand behavior by customer segment (age, gender, type).
- Analyze feedback and refund patterns to improve future service.
- Recommend improvements based on real patterns, not assumptions.

Scope:

- Analyze sales, customer, guide, refund, and feedback data for 2023–2024.

Deliverables:

- BigQuery SQL analysis queries
- Power BI dashboard with filters for customer/package dimensions
- KPI Summary page

- Feedback/refund insights by guide, package, customer
- Business recommendations document

7. Key KPIs:

- Total Bookings and Revenue (by month, by channel)
- Average Satisfaction Score (by package, by guide)
- Refund Rate and Volume (by reason, by month)
- Top Customer Segments (age group, type)
- Net Promoter Score (via `WouldRecommend` %)

8. Tools & Tech Stack:

- Google BigQuery (querying & transformation)
- Python (synthetic data generation & data quality checking)
- GCS (storage)
- Power BI (dashboard)
- GitHub (version control & documentation)
- Prezi (presentation)
- Airflow (Automation)

9. Timeline:

1. Business Requirements
2. Data Creation
3. Data Transformation
4. Data Validation / QA
5. Upload to GCS
6. Query Analysis
7. Data Dictionary / Docs
8. EDA
9. Power BI Dashboard
10. KPI Dashboard / Summary Page
11. Insight Log / Journal
12. Report Writing (Word/PDF)
13. Presentation Slide (PPT/Prezi)
14. Business Recommendations Slide
15. What-If / Scenario Modelling
16. Hypothesis Testing
17. Executive Summary Slide
18. GitHub Repository
19. DAG / Automation Plan
20. User Stories / Use Case Mapping

Limitations:

1. Data is synthetic and not reflect the original values related to company performance
2. Purposely to mirror the ongoing projects happen to improve focus and expand area that are crucial
3. Some parts cannot be implement due to data limitations and time constraints

DATA CREATION

Objective:

Optimizing Travel Package Performance, Customer Satisfaction Analysis & Data Collection enhancement.

Tools & Environment:

1. IDE: Visual Studio Code (VS Code)
2. Language: Python
3. Libraries Used: pandas, numpy, random, datetime, uuid

Methodology:

The data was created using custom Python scripts with the following structure:

1. Define schema: Identified columns and data types for the dataset (e.g., customer name, product, sales amount, transaction date).
2. Generate synthetic data: Used loops and libraries like random and numpy to generate realistic values.
3. Introduce variability: Added randomness to simulate real business data (e.g., seasonal sales spikes, missing values).
4. Save output: Exported the final dataset as a .csv file for ingestion into GCS.

Limitations:

1. Data is synthetic and not reflect the original values related to company performance
2. Data only available based on what authors decided to include
3. Some parts cannot be implemented due to data limitations and time constraints

1. Customers Table:

a) Sample codes:

https://github.com/nasrulkhair/Full-Project/blob/main/PPNJ/data_creation/data.ipynb

b) Output Description:

- File created: customers.csv
- Rows:
- Columns:

c) Table Schema

Column Name	Type	Description
customer_id	STRING	NON_NULLABLE
customer_name	STRING	NULLABLE
gender	STRING	NULLABLE
age	INTEGRER	NULLABLE
email	STRING	NULLABLE
customer_type	STRING	NULLABLE

2. Travel Package Table

a) Sample codes:

https://github.com/nasrulkhair/Full-Project/blob/main/PPNJ/data_creation/data.ipynb

b) Output Description:

- File created: travel_package.csv
- Rows:
- Columns:

c) Table Schema:

Column Name	Type	Description
package_id	STRING	NON_NULLABLE
package_name	STRING	NULLABLE
package_type	STRING	NULLABLE
duration_days	INTEGER	NULLABLE
price_per_pax	INTEGER	NULLABLE
category	STRING	NULLABLE
destination	STRING	NULLABLE

3. Travel Guide Table

a) Sample codes:

https://github.com/nasrulkhair/Full-Project/blob/main/PPNJ/data_creation/data.ipynb

b) Output Description:

- File created: travel_guide.csv
- Rows:
- Columns:

c) Table Schema:

Column Name	Type	Description
staff_id	INTEGER	NON_NULLABLE
staff_name	STRING	NULLABLE
role	STRING	NULLABLE
working_status	INTEGER	NULLABLE
comission_rate	FLOAT	NULLABLE

4. Sales Table

a) Sample codes:

https://github.com/nasrulkhair/Full-Project/blob/main/PPNJ/data_creation/data.ipynb

b) Output Description:

- File created: sales.csv
- Rows:
- Columns:

c) Table Schema:

Column Name	Type	Description
booking_id	STRING	NON_NULLABLE
customer_name	STRING	NULLABLE
package_id	STRING	NULLABLE
guide_id	INTEGER	NULLABLE
booking_date	DATE	NULLABLE
travel_date	DATE	NULLABLE
num_of_pax	INTEGER	NULLABLE
discount_promotion	INTEGER	NULLABLE
channel	STRING	NULLABLE
payment_status	STRING	NULLABLE
payment_method	STRING	NULLABLE

5. Feedback Table

a) Sample codes:

https://github.com/nasrulkhair/Full-Project/blob/main/PPNJ/data_creation/data.ipynb

b) Output Description:

- File created: feedback.csv
- Rows:
- Columns:

c) Table Schema:

Column Name	Type	Description
feedback_id	STRING	NON_NULLABLE
package_id	STRING	NULLABLE
booking_id	STRING	NULLABLE
satisfaction_score	INTEGER	NULLABLE
feedback_date	DATE	NULLABLE
comments	STRING	NULLABLE
rating_hotel	INTEGER	NULLABLE
rating_guide	INTEGER	NULLABLE
rating_itinerary	INTEGER	NULLABLE
would_recommend	STRING	NULLABLE

6. Refund Table

d) Sample codes:

https://github.com/nasrulkhair/Full-Project/blob/main/PPNJ/data_creation/data.ipynb

e) Output Description:

- File created: refund.csv
- Rows:
- Columns:

f) Table Schema:

Column Name	Type	Description
refund_id	STRING	NON_NULLABLE
booking_id	STRING	NULLABLE
customer_id	STRING	NULLABLE
refund_date	DATE	NULLABLE
refund_amount	INTEGER	NULLABLE
refund_percentage	FLOAT	NULLABLE
reason	STRING	NULLABLE
refund_status	STRING	NULLABLE

Transformations:

https://github.com/nasrulkhair/Full-Project/blob/main/PPNJ/data_creation/data.ipynb

No.	Table	Column	Transformation
1.	customers	email	fill null values with 'not provided'
2.	customers	email	replacement: , → " / ' : ' → ' '
3.	feedback	feedback_date	convert data type from STRING to DATE
4.	refund	refund_date	convert data type from STRING to DATE
5.	travel_guide	role	' ' → ''
6.	sales	payment_method	replace ' ' → ''

DATA TRANSFORMATION

Objective:

The goal of this phase was to take the raw/generated data and reshape it into a clean, analysis-ready format. Transformation taken involved:

- Ensuring consistent structure
- Renaming columns for clarity
- Reshaping or joining data as needed

Tools & Environment:

1. IDE: Visual Studio Code (VS Code)
2. Language: Python
3. Libraries Used: pandas, numpy, datetime

Transformation Activity:

1. Check data types and apply conversions as needed
2. Rename columns using lower case formatting.
3. Replace inconsistent values to ensure row-level consistency.
4. Handle null values by filling them based on agreed upon rules.

Limitations:

1. The data is synthetic and does not reflect actual values related to company performance.
2. There were minimal transformations applied in this project, as the data was intentionally generated in good condition for project purposes.

Transformation Workflow:

1. Load Raw Data:

- Loaded the raw csv files generated from the data creation phase

```
import pandas as pd
import numpy as np

customers = pd.read_csv('customers.csv')
travel_package = pd.read_csv('travel_package.csv')
travel_guide = pd.read_csv('travel_guide.csv')
sales = pd.read_csv('sales.csv')
feedback = pd.read_csv('feedback.csv')
refund = pd.read_csv('refund.csv')
```

2. Customers Table:

- Clean 'email' column by replacing values
- Fill the null values with 'not_provided'

```
# 1. customers table
# clean up the email column
customers['email'] = customers['email'].str.replace(';', '').str.replace('.,:')
customers.isna().sum()

# fill na email with 'not provided'
customers['email'] = customers['email'].fillna('not_provided')
```

Sample Final Output:

	customer_id	customer_name	gender	age	email	customer_type
0	MY1000	Zelaya Purwanti	male	36	zelaya.purwanti@outlook.com	new
1	MY1001	Dr. Mila Melani, M.Farm	female	29	dr.mila.melani.m.farm@hotmail.com	regular
2	MY1002	Cengkal Mulyani	female	43	cengkal.mulyani@yahoo.com	regular
3	MY1003	Siti Susanti	male	41	siti.susanti@yahoo.com	new
4	MY1004	R.M. Bakiono Hassanah, S.Pd	female	37	r.m.bakiono.hassanah.s.pd@yahoo.com	new

3. Feedback Table:

- feedback_date column is converted from string to date data type.

```
# 2. feedback table

feedback.dtypes

feedback['feedback_date'] = pd.to_datetime(feedback['feedback_date'])
feedback.head()
```

Sample Final Output

	feedback_id	package_id	booking_id	satisfaction_score	feedback_date	comments	rating_hotel	rating_guide	rating_itinerary	would_recommend
0	FB001	PC25	BK752	8	2023-07-11	Highly recommend this package to others.	3	5	5	Yes
1	FB002	PC10	BK617	10	2023-11-27	The hotel and guide exceeded our expectations!	5	4	5	Yes
2	FB003	PC31	BK677	10	2023-06-34	Highly recommend this package to others.	5	4	5	Yes
3	FB004	PC21	BK481	10	2024-02-29	The hotel and guide exceeded our expectations!	5	4	5	Yes
4	FB005	PC42	BK1109	8	2024-01-31	Beautiful destinations and a relaxing itinerary.	3	4	3	Yes

4. Refund Table:

- refund_date column is converted from string to date data type.

```
refund['refund_date'] = pd.to_datetime(refund['refund_date'])
refund.dtypes
```

Sample Final Output:

	refund_id	booking_id	customer_id	refund_date	refund_amount	refund_percentage	reason	refund_status
0	RF001	BK814	MY1522	2023-05-18	2245	9.95	Unforeseen circumstances	Pending
1	RF002	BK369	MY1592	2023-11-22	809	22.26	Health emergencies	Rejected
2	RF003	BK770	MY1452	2023-04-13	485	17.40	Other	Pending
3	RF004	BK704	MY1248	2023-09-23	1937	49.73	Natural disasters	Pending
4	RF005	BK1137	MY2163	2023-10-07	453	11.78	Unforeseen circumstances	Pending

5. Travel Guide Table:

- Clean 'role' column by replacing values

```
# 4. travel_guide table

travel_guide['role'] = travel_guide['role'].str.replace('_', ' ')
travel_guide.head()
```

Sample Final Output:

	staff_id	staff_name	role	working_status	comission_rate
0	0101	Nasrul	main guide	contract	0.08
1	0102	Khair	assistant guide	contract	0.11
2	0103	Intan	assistant guide	freelance	0.12
3	0104	Amira	main guide	freelance	0.12
4	0105	Nuha	main guide	contract	0.08

6. Travel Package Table:

- No transformation done.

Sample Final Output:

	package_id	package_name	package_type	duration_days	price_per_pax	category	destination
0	PC10	Manila	solo	6	2387	adventure	Manila
1	PC11	Krabi	group	4	2256	eco	Krabi
2	PC12	Taman Negara	solo	2	1018	adventure	Taman Negara
3	PC13	Putrajaya	family	3	763	adventure	Putrajaya
4	PC14	Cebu	family	5	1784	cultural	Cebu

7. Sales Table:

- Clean 'payment_method' column by replacing values

```
sales['payment_method'] = sales['payment_method'].str.replace('_', ' ')
sales.head()
```

Sample Final Output:

	booking_id	customer_id	package_id	guide_id	booking_date	travel_date	num_of_pax	discount_promotion	channel	payment_status	payment_method
0	BK100	MY1525	PC37	0105	2023-06-17	2023-07-10	2	645	online	paid	credit card
1	BK101	MY1108	PC27	0109	2023-08-13	2023-09-11	4	0	offline	paid	online transfer
2	BK102	MY2042	PC18	0106	2023-06-26	2023-08-24	17	1895	online	paid	cash
3	BK103	MY1263	PC26	0101	2023-12-27	2024-01-18	4	230	agent	pending	credit card
4	BK104	MY1229	PC32	0106	2023-04-23	2023-06-19	12	0	agent	paid	cash

8. Saving Outputs: "path: End To End Project/PPNJ/data creation"

```
# Saving all csv files
customers.to_csv(r"C:\Users\User\Desktop\Data Analyst\End To End
Project\PPNJ\data_creation\dim_customer.csv", index=False)

travel_package.to_csv(r"C:\Users\User\Desktop\Data Analyst\End To End
Project\PPNJ\data_creation\dim_travel_package.csv", index=False)

sales.to_csv(r"C:\Users\User\Desktop\Data Analyst\End To End Project\PPNJ\data_creation\fact_sales.csv",
index=False)
feedback.to_csv(r"C:\Users\User\Desktop\Data Analyst\End To End
Project\PPNJ\data_creation\dim_feedback.csv", index=False)

travel_guide.to_csv(r"C:\Users\User\Desktop\Data Analyst\End To End
Project\PPNJ\data_creation\dim_travel_guide.csv", index=False)

refund.to_csv(r"C:\Users\User\Desktop\Data Analyst\End To End Project\PPNJ\data_creation\dim_refund.csv",
index=False)
```

Summary of Transformations:

https://github.com/nasrulkhair/Full-Project/blob/main/PPNJ/data_creation/data.ipynb

- All transformations were done programmatically using pandas to ensure reproducibility.
- Output is clean, schema-consistent, and ready for further modelling

No.	Table	Column	Transformation
1.	customers	email	fill null values with 'not provided'
2.	customers	email	replacement: , → "/" : → "
3.	feedback	feedback_date	convert data type from STRING to DATE
4.	refund	refund_date	convert data type from STRING to DATE
5.	travel_guide	role	'_ ' → ""
6.	sales	payment_method	replace '_ ' → ""

Data Validation

Objective:

Ensure that the synthetic dataset is logically sound, clean, and matches real-world expectations based on defined business rules and data dictionary specifications.

Validation Type	Description	Result
Null Checks	Checked for unexpected nulls in key fields (e.g., customer_id, booking_date, package_name)	 Passed
Data Type Checks	Ensured correct data types for all fields (e.g., age is integer, price_per_pax is float)	 Passed
Value Ranges	Verified ranges (e.g., age between 18–70, satisfaction_score between 1–10)	 Passed
Date Logic	Confirmed booking_date < travel_date, and refund_date falls between booking/travel	 Passed
Duplicate Check	Ensured no duplicate booking_id, customer_id, package_id in main tables	 Passed
Referential Integrity	All foreign key relationships (e.g., customer, package, guide IDs) resolve correctly	 Passed
Business Logic Rules	Validated: only refunded records have refund_status, only future travel = pending payment	 Passed
Skew Distribution	Verified skew in distributions (e.g., majority customers aged 25–40, higher ratings, low refund %)	 Passed

Data Validation Sample Codes:

```
# Null check
df.isnull().sum()

# Age range
assert df['age'].between(18, 70).all()

# Booking vs travel date
assert (df['booking_date'] < df['travel_date']).all()

# Refund logic
assert df[df['refund_status'].notnull()]['payment_status'].eq('refunded').all()
```


Google Cloud Storage (GCS)

Objective:

To ensure the transformed datasets are securely accessible and ready for further use, this step involves uploading them to a Google Cloud Storage (GCS) bucket. The key goals include:

- **Data Security:** Ensure that the uploaded datasets are protected through proper access controls, encryption at rest and in transit, and IAM policies that restrict access to authorized users only
- **Integration with Google Cloud Ecosystem:** Enable seamless connectivity with other Google services, such as BigQuery for advanced analytics, Looker Studio for visual reporting, and Vertex AI for machine learning applications.
- **Pipeline Enablement:** Support end-to-end data pipelines by making datasets available for:
 - Business intelligence dashboards (Power Bi)
 - Scheduled updates or data refreshes through automated workflow.

Tools & Configuration:

1. **Platform:** Google Cloud Platform (GCP)
2. **Service:** Cloud Storage (GCS)
3. **Authentication:** GCP service account key (JSON)
4. **Upload Method:** Python script using google-cloud-storage SDK
5. **Destination Bucket:** gs://ppnj_data/files /

Process Overview:

1. Authenticate using a service account key (credentials.json).
2. Initialize GCS client.
3. Define destination path in the GCS bucket.
4. Upload files in the list variable:
 - fact_sales.csv
 - dim_customers
 - dim_travel_package.csv
 - dim_travel_guide.csv
 - dim_feedback.csv
 - dim_refund.csv
5. Verify upload by listing objects in the bucket

Limitations:

The Google Cloud Platform (GCP) used in this project is owned by the project author and is not the official GCP account of the company

Sample Upload Script (Python)

```
import os
from google.cloud import storage

# path to GCP credentials
os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = r"C:\Users\User\Desktop\GCP\ppnj-463412-13c39ea0960c.json"

# function to upload files to GCS
def upload_to_gcs(bucket_name, source_file, destination_blob):
    client = storage.Client()
    bucket = client.bucket(bucket_name)
    blob = bucket.blob(destination_blob)

    blob.upload_from_filename(source_file)
    print(f"Uploaded {source_file} to gs://{bucket_name}/{destination_blob}")

# Files to upload
files_to_upload = [
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End Project\PPNJ\data_creation\dim_customer.csv",
        "destination_blob": "files/dim_customer.csv"
    },
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End Project\PPNJ\data_creation\dim_feedback.csv",
        "destination_blob": "files/dim_feedback.csv"
    },
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End Project\PPNJ\data_creation\dim_refund.csv",
        "destination_blob": "files/dim_refund.csv"
    },
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End Project\PPNJ\data_creation\fact_sales.csv",
        "destination_blob": "files/fact_sales.csv"
    },
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End Project\PPNJ\data_creation\dim_travel_guide.csv",
        "destination_blob": "files/dim_travel_guide.csv"
    },
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End Project\PPNJ\data_creation\dim_travel_package.csv",
        "destination_blob": "files/dim_travel_package.csv"
    }
]

# Upload all files to GCS
for file in files_to_upload:
    upload_to_gcs("ppnj_data", file['source_file'], file['destination_blob'])
```

Files Uploaded to GCS

File Name	GCS Path
fact_sales.csv	gs://ppnj_data/files/fact_sales.csv
dim_customers.csv	gs://ppnj_data/files/dim_customer.csv
dim_travel_package.csv	gs://ppnj_data/files/dim_travel_package.csv
dim_travel_guide.csv	gs://ppnj_data/files/dim_travel_guide.csv
dim_feedback.csv	gs://ppnj_data/files/dim_feedback.csv
dim_refund.csv	gs://ppnj_data/files/dim_refund.csv

Summary:

1. All files are stored in GCS with private access by default.
2. It can be easily connected to BigQuery, Data Studio, or external dashboards.
3. Upload is fully scripted and repeatable, supporting automation.

Query Analysis Report

Objective:

To extract meaningful insights from the datasets using structured SQL queries, each aligned with a specific business goal. Queries were designed to answer both strategic and tactical questions from multiple perspectives. The goal was to answer business-critical questions across four core themes:

1. Business Questions
2. Customer Behavior
3. Customer Feedback
4. Sales Insight

All queries were executed in Google BigQuery, using structured, documented SQL logic. Each query is designed to drive specific visuals or KPIs in the final dashboard.

Tools & Environment:

1. **Database:** Google BigQuery
2. **Query Language:** SQL
3. **Editor:** BigQuery Console
4. **Source Tables:**
 - dim_date
 - fact_sales
 - dim_customers
 - dim_travel_package
 - dim_travel_guide
 - dim_feedback
 - dim_refund

SQL Files Overview:

File Name	Description
dim_date.sql	Date dimension table creation.
Business Questions.sql	High-level business questions & KPIs
Customer Behavior Insight Framework.sql	Insights on customer type, age, loyalty
Customer Feedback Analysis.sql	Sentiment, recommendation, and guide performance
Sales Insight Framework.sql	Trends in revenue, bookings, discounts

Analysis:

1. Business Questions

Business Problem: Address management-level KPI and strategic decisions.

#	Question	SQL Query	Insights Summary
1.	Top 3 Travel Packages by Monthly Revenue.	<pre>SELECT * FROM `ppnj.dim_date`; WITH monthly_revenue AS (SELECT d.month month, d.year year, tp.package_name, SUM(s.num_of_pax * tp.price_per_pax - s.discount_promotion) total_revenue FROM `ppnj.dim_date` d JOIN `ppnj.fact_sales` s ON d.date_key = s.travel_date JOIN `ppnj.dim_travel_package` tp ON s.package_id = tp.package_id GROUP BY d.year, d. month, tp.package_name), ranked_revenue AS (SELECT *, RANK() OVER(PARTITION BY package_name ORDER BY total_revenue DESC) package_rank FROM monthly_revenue) SELECT * FROM ranked_revenue WHERE package_rank <=3;</pre>	- Nha Trang emerged as the top revenue generator, reaching RM 33, 736 in October 2023. - Vientiane appeared 3 times in the Top 3 across May, July, and November 2023, with a consistent revenue range around RM 23,477 to RM 28,597. - Jakarta made the Top 3 in June, July, and September 2023, indicating a strong mid-year performance. - Putrajaya remained competitive in August 2023 and January 2024, showing potential in domestic travel offerings. # Visuals Recommendation: - Line Chart (small multiples)
2.	Average Discount per Channel and Usage Rate	<pre>SELECT channel, COUNT(*) total_bookings, ROUND(AVG(discount_promotion), 2) avg_discount, SUM(CASE WHEN discount_promotion > 0 THEN 1 ELSE 0 END) discounted_bookings, ROUND(SUM(CASE WHEN discount_promotion > 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) dicount_usage_rate FROM `ppnj.fact_sales` GROUP BY channel;</pre>	- Walk-in customers received the highest average discount of RM 456.09, and the highest discount usage rate 61.98%, despite having fewest total bookings of 121. - Across all channels, discount usage rates were relatively consistent around 60%, suggesting uniform promotion strategies.
3.	High-Value One-Time Customers	<pre>WITH customer_spending AS (SELECT c.customer_id, c.customer_name, c.customer_type, c.age, COUNT(s.booking_id) AS total_bookings, SUM(s.num_of_pax * tp.price_per_pax - s.discount_promotion) AS total_spent FROM `ppnj.fact_sales` s JOIN `ppnj.dim_customers` c ON s.customer_id = c.customer_id JOIN `ppnj.dim_travel_package` tp ON s.package_id = tp.package_id GROUP BY c.customer_id, c.customer_name, c.customer_type, c.age) SELECT * FROM customer_spending WHERE total_bookings = 1 ORDER BY total_spent DESC;</pre>	- All are categorized as regular customers, with ages ranging between 18 and 40 years. - The highest spender with RM 38, 116, only booked once and did not return. - A few customers represent a younger high-value that may be worth targeting in retention campaigns # Visual Recommendation: Highlight table with customer name, age, total spent, and a visual tag for customer type. Optional: Use a funnel or retention chart showing drop-off after 1st high-value trip.

4.	Top 5 Guides with Lowest Ratings	<pre> SELECT g.staff_name, ROUND(AVG(fb.rating_guide), 2) AS avg_guide_rating, COUNT(DISTINCT r.refund_id) AS total_refunds, COUNT(fs.booking_id) AS total_bookings FROM `ppnj.fact_sales` fs JOIN `ppnj.dim_travel_guide` g ON fs.guide_id = g.staff_id LEFT JOIN `ppnj.dim_feedback` fb ON fs.booking_id = fb.booking_id LEFT JOIN `ppnj.dim_refund` r ON fs.booking_id = r.booking_id GROUP BY g.staff_name ORDER BY avg_guide_rating ASC LIMIT 5; </pre>	- The bottom 5 guides received average ratings between 3.13 and 3.55, which is significantly lower than the expected service standard. - These results highlight potential service quality concerns, customer dissatisfaction, or mismatches in customer expectations. # Visual Recommendation - A horizontal bar chart comparing average guide ratings. - A heatmap table with conditional formatting to flag guides with both low ratings and high refund counts.
5.	Refund Rate Among Low Satisfaction Customers	<pre> WITH low_feedback AS (SELECT booking_id FROM `ppnj.dim_feedback` WHERE satisfaction_score <= 3), refund_stats AS (SELECT COUNT(*) AS low_feedback_count, SUM(CASE WHEN r.booking_id IS NOT NULL THEN 1 ELSE 0 END) AS refunded_count FROM low_feedback lf LEFT JOIN `ppnj.dim_refund` r ON lf.booking_id = r.booking_id) SELECT low_feedback_count, refunded_count, ROUND(refunded_count * 100.0 / low_feedback_count, 2) AS refund_rate FROM refund_stats; </pre>	- A total of 28 bookings were flagged with low satisfaction scores. - None of these bookings resulted in a refund request. - The refund rate among dissatisfied customers is 0.0%. # Visual Recommendation - A KPI card showing “0.0% refund rate among dissatisfied customers”. - Supplement with a donut chart visualizing low-satisfaction customers by refund outcome.
6.	Top 3 Month of Peak revenue	<pre> WITH revenue_by_month AS (SELECT d.month month, d.monthname monthname, COUNT(s.booking_id) total_bookings, SUM(s.num_of_pax * tp.price_per_pax - s.discount_promotion) total_gross_revenue FROM `ppnj.fact_sales` s JOIN `ppnj.dim_date` d ON s.booking_date = d.date_key JOIN `ppnj.dim_travel_package` tp ON s.package_id = tp.package_id GROUP BY d.month, d.monthname), rank_month AS (SELECT *, RANK() OVER(ORDER BY total_gross_revenue DESC) month_rank FROM revenue_by_month) SELECT * FROM rank_month WHERE month_rank <= 3; </pre>	- August is the top-performing month, generating the highest gross revenue despite not having the highest number of bookings. - June and July follow closely in revenue, showing strong performance in the mid-year period. - This suggests that Q3 (July–September) is the peak travel season, possibly influenced by school holidays, festive periods, or marketing efforts. # Visual Recommendation - Bar chart showing gross revenue per month (highlighting top 3). - Optionally, add a dual-axis line chart showing both revenue and number of bookings for better context.

2. Customer Behaviour Insight Framework

Business Problem: Discover customer characteristics and booking habits.

#	Question	SQL Query	Insights Summary
1.	Customer Demographic Profile	<pre>SELECT gender, customer_type, COUNT(customer_id) total_customers, ROUND(AVG(age),0) avg_age FROM `ppnj.dim_customers` GROUP BY gender, customer_type;</pre>	- The customer base is evenly distributed between males and females, with a slight majority of regular female customers (420). - The average age across all segments falls within a narrow range (34–36 years), indicating a consistent age profile of customers. - The "Other" gender category has relatively low representation (55 customers total), but with age patterns consistent with the other groups.
2.	Preferred Package Name by Traveller type	<pre>SELECT tp.package_name, tp.package_type, COUNT(s.booking_id) total_booking FROM `ppnj.fact_sales` s JOIN `ppnj.dim_travel_package` tp ON s.package_id = tp.package_id JOIN `ppnj.dim_customers` c ON s.customer_id = c.customer_id GROUP BY tp.package_type, tp.package_name;</pre>	- Family and honeymoon packages dominate in booking volume. - Traveller types exhibit clear destination preferences, opening opportunities for custom itineraries or bundled deals. - Solo travellers lean toward accessible cities, while honeymooners seek beach and island packages.
3.	Repeat vs. New Customers	<pre>SELECT CASE WHEN booking_count > 1 THEN 'repeat' ELSE 'new' END customer_status, COUNT(*) num_customers FROM (SELECT customer_id, COUNT(*) booking_count FROM `ppnj.fact_sales` GROUP BY customer_id) sub GROUP BY customer_status;</pre>	- A majority (57.7%) of the customer base are new customers, suggesting strong top-of-funnel acquisition. - However, a healthy 42.3% have returned for multiple bookings, indicating positive customer experience and brand trust. - The split also shows potential to grow loyalty further by nurturing first-time customers into repeat clients.
4.	Booking Lead Time by Customer Type	<pre>SELECT tp.package_type, ROUND(AVG(DATE_DIFF(s.travel_date, s.booking_date, DAY)), 0) avg_lead_days FROM `ppnj.fact_sales` s JOIN `ppnj.dim_travel_package` tp ON s.package_id = tp.package_id GROUP BY tp.package_type;</pre>	- Across all package types, customers book approximately 6 weeks in advance. - Solo travelers book slightly earlier than others — an average of 46 days in advance — possibly reflecting more flexibility and early planning behavior. - Family, group, and honeymoon travelers book 43–44 days ahead, likely due to the need for coordination among multiple people or event timing (e.g., honeymoons).

5.	Top 1 preference by age group for each destination	<pre> WITH age_group_bookings AS (SELECT CASE WHEN c.age < 25 THEN 'under-25' WHEN c.age BETWEEN 25 AND 34 THEN '25-34' WHEN c.age BETWEEN 35 AND 49 THEN '35-49' ELSE '50+' END age_group, tp.destination, COUNT(s.booking_id) total_booking FROM `ppnj.fact_sales` s JOIN `ppnj.dim_travel_package` tp ON s.package_id = tp.package_id JOIN `ppnj.dim_customers` c ON s.customer_id = c.customer_id GROUP BY age_group, tp.destination), ranked_booking AS (SELECT *, ROW_NUMBER() OVER(PARTITION BY destination ORDER BY total_booking DESC) rank_destination FROM age_group_bookings) SELECT age_group, destination, total_booking FROM ranked_booking WHERE rank_destination = 1; </pre>	- The 35–49 age group is the dominant traveler segment for the majority of destinations. - The 25–34 age group leads bookings for some popular and adventurous locations, such as: • Cameron Highlands (21 bookings) • Langkawi (14) • Pulau Redang (13) • Ho Chi Minh City (13) • Yogyakarta (19) - The under-25 and 50+ age groups did not emerge as top travelers for any destination in this analysis, suggesting they may be smaller market segments. - Travelers aged 35–49 appear to be the primary audience, especially for destinations like: • Borneo, Dili, and Bangkok - Destinations favored by 25–34-year-olds tend to be nature-based or offbeat urban escapes. # Visual Recommendations: - Heatmap or matrix chart: Age group vs. Destination with booking volume. - Bar chart showing top destinations for 25–34 and 35–49 age groups side by side.
6.	Customer Channel Preferences	<pre> WITH age_group_booking AS (SELECT CASE WHEN c.age < 25 THEN 'under-25' WHEN c.age BETWEEN 25 AND 34 THEN '25-34' WHEN c.age BETWEEN 35 AND 49 THEN '35-49' ELSE '50+' END age_group, s.channel, COUNT(s.booking_id) total_booking FROM `ppnj.fact_sales` s JOIN `ppnj.dim_customers` c ON s.customer_id = c.customer_id GROUP BY s.channel, age_group), rank_by_age_group AS (SELECT *, ROW_NUMBER() OVER(PARTITION BY age_group ORDER BY total_booking DESC) rn FROM age_group_booking) SELECT age_group, channel, total_booking FROM rank_by_age_group WHERE rn = 1; </pre>	- All age groups — including seniors (50+) — predominantly use the online channel for bookings. - The 35–49 group is the most active online, followed by 25–34. - Online is the dominant booking channel across all age segments. - Despite assumptions that older generations may prefer offline methods, the 50+ group also prefers online bookings, though at a lower volume. # Visual Recommendations: - Matrix chart: Age group vs. channel with booking volume. - Stacked bar chart: Total bookings segmented by age group and channel.

3. Customer Feedback Analysis

Business Problem: Understand satisfaction, loyalty, and refund behavior.

#	Questions	SQL Query	Insights Summary
1.	Average Satisfaction Score by Package	<pre>SELECT tp.package_name, COUNT(f.feedback_id) num_feedback, ROUND(AVG(f.satisfaction_score), 2) avg_score FROM `ppnj.dim_travel_package` tp JOIN `ppnj.dim_feedback` f ON tp.package_id = f.package_id GROUP BY tp.package_name ORDER BY avg_score DESC;</pre>	- The highest-rated package is Yangon with an average score of 8.33, suggesting very strong customer satisfaction. - Pulau Sipadan and Vientiane also deliver exceptional customer experiences. - At the other end, Ho Chi Minh City, Phuket, and Dili have the lowest scores, indicating a need for improvement.
2.	Ratings Breakdown by Travel Guide	<pre>SELECT tg.staff_name, ROUND(AVG(f.rating_guide), 2) avg_guide_rating, COUNT(f.feedback_id) total_reviews FROM `ppnj.dim_travel_guide` tg JOIN `ppnj.fact_sales` s ON s.guide_id = tg.staff_id JOIN `ppnj.dim_feedback` f ON s.booking_id = f.booking_id GROUP BY tg.staff_name ORDER BY avg_guide_rating DESC;</pre>	- Nasir is the highest-rated guide with an average of 3.77, followed closely by Nasrul and Alissa. - Intan has the lowest rating at 3.13, which may indicate a need for performance review or customer engagement improvement. - Most guide ratings hover between 3.1 to 3.8, indicating overall moderate satisfaction, but with room for improvement.
3.	Would Recommend Rate	<pre>SELECT ROUND(SUM(CASE WHEN recommend_group = 'Yes' THEN num_feedback ELSE 0 END) * 100.0 / SUM(num_feedback), 2) AS yes_percentage FROM (SELECT CASE WHEN would_recommend THEN 'Yes' ELSE 'No' END AS recommend_group, COUNT(*) AS num_feedback FROM `ppnj.dim_feedback` GROUP BY recommend_group);</pre>	- Nearly 8 out of 10 customers (79.38%) would recommend the company's services to others. - This suggests a high level of satisfaction, likely driven by good experiences with packages, customer service, or pricing. - However, the remaining ~21% detractors are still significant and warrant investigation into common pain points. # Visual Recommendation: - Donut Chart: Display percentage split between "Yes" and "No".
4.	Refund Rate by Package	<pre>SELECT p.package_name, COUNT(r.refund_id) AS total_refunds, COUNT(fs.booking_id) AS total_bookings, ROUND(COUNT(r.refund_id) * 100.0 / COUNT(fs.booking_id), 2) AS refund_rate FROM `ppnj.fact_sales` fs JOIN `ppnj.dim_travel_package` p ON fs.package_id = p.package_id LEFT JOIN `ppnj.dim_refund` r ON fs.booking_id = r.booking_id GROUP BY p.package_name ORDER BY refund_rate DESC;</pre>	- Bagan tops the list with a 12% refund rate, a red flag for service quality or experience issues. - Manila and Krabi follow closely with 8.7% each, warranting immediate attention. - Several packages such as Chiang Mai, Langkawi, and Melaka have moderate refund rates between 5% to 7%. - Encouragingly, over 40% of packages (17 out of 39) have zero refunds, indicating strong operational and customer satisfaction performance.
5.	Refund Amount & Reason for refund	<pre>SELECT refund_status, COUNT(*) AS num_refunds, SUM(refund_amount) AS total_refund_value, ROUND(AVG(refund_percentage), 2) AS avg_percent FROM `ppnj.dim_refund` GROUP BY refund_status;</pre>	- Approved refunds dominate both in count and value, making up over 55% of total refunds. - Even pending and rejected refunds carry significant amounts, signalling unresolved issues that still impact customer sentiment.

4. Sales Insight Framework

Business Problem: Explore trends in bookings, revenue, and promotions.

#	Questions	SQL Query	Insights Summary
1.	Total Bookings & Revenue Over Time	<pre>SELECT d.year, d.monthname, COUNT(fs.booking_id) total_booking, SUM(fs.num_of_pax * tp.price_per_pax - fs.discount_promotion) total_revenue, ROUND(AVG(fs.num_of_pax * tp.price_per_pax - fs.discount_promotion), 2) avg_revenue_per_booking FROM `ppnj.fact_sales` fs JOIN `ppnj.dim_date` d ON fs.booking_date = d.date_key JOIN `ppnj.dim_travel_package` tp ON fs.package_id = tp.package_id GROUP BY d.year, d.monthname ORDER BY d.year, d.monthname;</pre>	- August recorded the highest total revenue at RM 1,155,382, driven by 138 bookings. - July had the highest number of bookings at 147, but slightly lower revenue (RM 1,063,219) than August — possibly due to cheaper packages or more discounts applied. - March and November stood out with the highest average revenue per booking at RM 9,453.30 and RM 9,560.54 respectively, despite lower booking volumes. - May to July saw consistent growth in bookings, showing strong mid-year performance.
2.	Top Performing Travel Packages	<pre>SELECT tp.package_name, COUNT(s.booking_id) total_booking, SUM(s.num_of_pax * tp.price_per_pax) total_revenue FROM `ppnj.fact_sales` s JOIN `ppnj.dim_travel_package` tp ON s.package_id = tp.package_id GROUP BY tp.package_name ORDER BY total_revenue DESC;</pre>	- Hanoi leads with RM 1,120,230 in total revenue, generated from 40 bookings — suggesting either high package pricing, larger group sizes, or both. - Phnom Penh and Krabi made it into the top 5 with fewer bookings (34 and 23, respectively), implying these are high-value packages likely popular with premium or international travellers. - Cameron Highlands had the most bookings (50) but appears lower on revenue rank (RM 133,680) — suggesting it's a budget-friendly or domestic favorite package.
3.	Channel Performance Analysis	<pre>SELECT s.channel, COUNT(s.booking_id) total_booking, SUM(tp.price_per_pax * s.num_of_pax) total_revenue FROM `ppnj.fact_sales` s JOIN `ppnj.dim_travel_package` tp ON s.package_id = tp.package_id GROUP BY s.channel ORDER BY total_revenue DESC;</pre>	- The online channel clearly dominates with 613 bookings and generating RM 5.18 million in total revenue. - This reflects a strong customer preference for digital booking platforms and possibly effective online marketing strategies.
4.	Discount Impact on Revenue	<pre>SELECT CASE WHEN s.discount_promotion > 0 THEN 'discounted' ELSE 'non-discounted' END AS discount_group, COUNT(s.booking_id) AS bookings, SUM(tp.price_per_pax * s.num_of_pax) gross_revenue, SUM(discount_promotion) total_discounted, SUM(tp.price_per_pax * s.num_of_pax - s.discount_promotion) net_revenue FROM `ppnj.fact_sales` s JOIN `ppnj.dim_travel_package` tp ON s.package_id = tp.package_id GROUP BY discount_group;</pre>	- Discounted bookings account for 748 bookings, generating a gross revenue of RM 6.3 million. - After applying RM 457.95k in discounts, the net revenue is still strong at RM 5.85 million. - This suggests that discounts successfully attract higher sales volume, compensating for the reduced margins. - non-discounted bookings are fewer (486 bookings), they still generated RM 4.12 million, with zero revenue loss from promotions. - Discounted bookings contribute 58% of total bookings and 59% of net revenue, showing a balanced impact.

5.	Revenue by Travel Month	<pre> SELECT d.monthname, SUM(s.num_of_pax * tp.price_per_pax - s.discount_promotion) revenue FROM `ppnj.fact_sales` s JOIN `ppnj.dim_travel_package` tp ON s.package_id = tp.package_id JOIN `ppnj.dim_date` d ON s.travel_date = d.date_key GROUP BY d.month, d.monthname ORDER BY d.month ASC; </pre>	- With RM 1.15 million in revenue, September generated the highest monthly travel revenue, indicating a strong seasonal demand—possibly due to school holidays or local events - June (RM 907k), July (RM 886k), and August (RM 920k) also showed high revenue, reinforcing the mid-year period as a prime travel season for targeted promotions. - February (RM 572k) and March (RM 715k) recorded the lowest revenues, suggesting off-peak months with lower demand—ideal for running aggressive marketing campaigns or flash deals. - November and December maintained healthy revenue levels, possibly linked to year-end holidays and Christmas break, though not as strong as mid-year or September.
----	-------------------------	--	--

Techniques Used:

- Window Functions: RANK(), ROW_NUMBER()
- CTEs: For breakdown of logic and reusability
- Aggregations: SUM(), AVG(), COUNT()
- CASE Statements: For conditional groupings
- JOINS: Across dimensions and fact tables
- QA Checks: For null or placeholder values

Output Usage:

Query results power visuals in Power BI, including:

- Revenue & booking trends
- Loyalty & churn charts
- Guide performance metrics
- Refund dashboards
- Segment-specific insights

Summary Conclusion:

- Structured SQL queries were used to answer strategic and operational business questions across four themes: Business Questions, Customer Behavior, Customer Feedback, and Sales Insight.
- Analysis revealed Q3 (July–September) as the peak travel season, with August and September showing the highest revenue.
- Online booking is the dominant channel across all age groups, contributing over RM 5 million in total revenue.
- Identified high-value one-time customers — key targets for retention campaigns.
- Hanoi, Phnom Penh, and Krabi emerged as top-revenue travel packages, indicating demand for premium experiences.
- Discounts are widely used (applied in 60%+ of bookings), proving effective in boosting booking volume without significantly harming revenue.
- Customer demographics are balanced by gender and age, with the 35–49 group being the most active travellers.
- Repeat customers make up 42.3%, showing potential for loyalty growth.
- Feedback analysis showed Yangon as the highest-rated destination, while Bagan and Dili had higher refund or lower satisfaction rates.
- Guide performance varied, with a few guides needing attention due to low ratings and higher refund rates.
- Refund rate among dissatisfied customers was 0%, suggesting potential disconnect between dissatisfaction and refund policy awareness.
- Insights will directly power Power BI dashboards, enabling data-driven tracking of KPIs, customer trends, and operational performance.

Data Dictionary

Purpose:

The Data Dictionary provides detailed documentation of the fields (columns) across the synthetic dataset. It also identifies data generation issues and describes how each was fixed or improved to enhance realism, accuracy, and usability in analysis.

No. Field / Issue		Description / Fix Applied
1	Flat gender / customer_type	Added weighted probabilities for more realistic gender and customer_type balance
2	Age random between 18–70	Applied a skewed normal distribution, concentrating around common traveling ages (25–40)
3	All emails perfectly generated	Introduced missing values and typos to simulate real-world inconsistencies
4	Names may not match gender	Assigned gender-specific names to maintain consistency
5	No duplicates or noise	Injected optional dirty rows and random noise (e.g., weird characters or blanks)
6	package_name	Ensured no duplicates — each destination is used only once
7	package_type	Made type relevant to destination (e.g., beachy places = honeymoon)
8	price_per_pax	Balanced prices across realistic travel tiers (budget, mid, premium)
9	duration_days	Longer durations for premium/far destinations
10	category	Made category align better with type/destination (e.g., hiking = adventure)
11	role	Used logic to assign fewer main_guides, more assistant_guides
12	working_status	Used weighted probabilities — “permanent” is most common
13	commission_rate	Kept existing logic (already realistic)
14	booking_date	Distributed bookings based on seasonality (e.g., school holidays = peak)

No. Field / Issue	Description / Fix Applied
15 travel_date	Always occurs after booking (1–90 day window)
16 discount_promotion	Skewed distribution — most users get 0–5%, only a few get high discounts
17 payment_status	Tied to travel_date — if in future = pending
18 payment_method	Weighted common methods higher — credit card & online dominate
19 SatisfactionScore	Skewed towards positive but with variance
20 Rating_guide, hotel, itinerary	Based on satisfaction ± small random variation
21 Comments	Tone matches sentiment of score (positive = good tone)
22 WouldRecommend	Already solid — left as-is
23 refund_rows	Only included bookings where payment_status = 'refunded'
24 refund_amount	Calculated as price × pax × refund %
25 refund_percentage	Tied to refund reason (e.g., emergency = higher %)
26 refund_date	Occurs after booking but before travel (or shortly after travel)
27 refund_status	Based on timing or randomness (e.g., approved = early + good reason)

Exploratory Data Analysis

Objective:

To explore the dataset for key trends, outliers, distributions, and data quality issues before building the Power BI dashboard and summarizing insights.

Data Quality Overview

Sample Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Loading private GCS files
customers = pd.read_csv("gs://ppnj_data/files/dim_customer.csv")
travel_package = pd.read_csv("gs://ppnj_data/files/dim_travel_package.csv")
sales = pd.read_csv("gs://ppnj_data/files/fact_sales.csv")
travel_guide = pd.read_csv("gs://ppnj_data/files/dim_travel_guide.csv")
feedback = pd.read_csv("gs://ppnj_data/files/dim_feedback.csv")
refund = pd.read_csv("gs://ppnj_data/files/dim_refund.csv")
```

Results:

Dataset	Rows	Key Notes
dim_customers	1234	No missing IDs or names, but email has 43 missing values (~3.5%). Consider cleaning/handling these for campaigns or contact flows.
dim_travel_package	39	Clean and complete. All packages have valid details including category, destination, and price.
fact_sales	1234	No missing values. Booking and travel dates are strings — should be converted to datetime.
dim_feedback	320	Full feedback entries, no missing scores. would_recommend is stored as string instead of boolean — check consistency.
dim_refund	30	Small subset of data — only 30 refunds. Clean and complete, but refund_percentage should be validated for outliers.

Null & Duplicate Checks

Sample Code:

```
# Checking for nulls

for name, df in {'Customers': customers, 'Sales': sales, 'Feedback': feedback, 'Refund': refund}.items():
    print(f"\n{name} missing values:")
    print(df.isnull().sum())

# Checking for duplicate booking_id or customer_id
print("Duplicate booking_id:", sales.duplicated('booking_id').sum())
print("Duplicate customer_id:", customers.duplicated('customer_id').sum())
```

Results:

1. Missing Values:

Table	Column(s) with Nulls	Count	Notes
Customers	email	43	~3.5% of customer records are missing an email. The missing data will be replaced with 'not provided' as agreed upon.
Others	<i>None</i>	0	All other key datasets (Sales, Feedback, Refund) are complete.

2. Duplicate Checks:

Table	Column Checked	Duplicates	Notes
Sales	booking_id	0	Each booking is uniquely recorded.
Customers	customer_id	0	Each customer is uniquely identified.

Key Distributions

Sample Code:

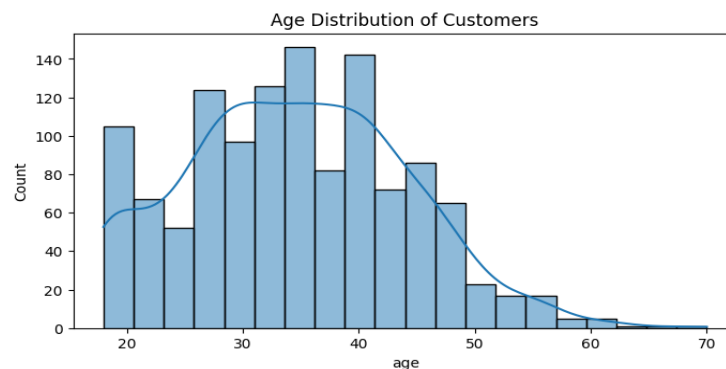
```
# Age distribution
plt.figure(figsize=(8,4))
sns.histplot(customers['age'], kde=True, bins=20)
plt.title("Age Distribution of Customers")
plt.show()

# Satisfaction score
sns.histplot(feedback['satisfaction_score'], bins=10, color='red')
plt.title("Satisfaction Score Distribution")
plt.show()

# Refund %
sns.histplot(refund['refund_percentage'], bins=10, color='skyblue')
plt.title("Refund Percentage Distribution")
plt.show()
```

Results:

1. Ages Distribution of Customers:



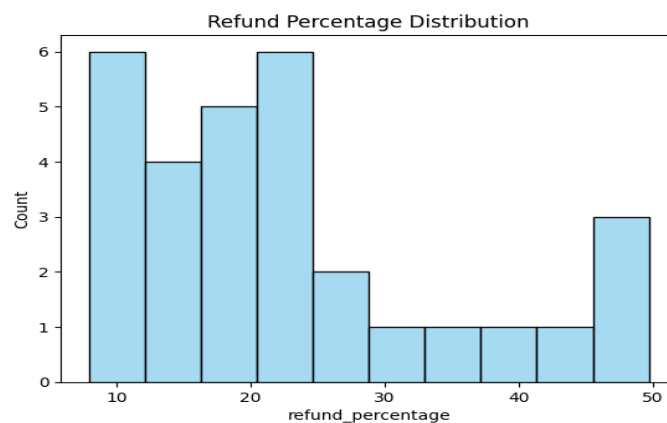
- The age of customers follows a right-skewed distribution, peaking around the 30–40 age group.
- There is a noticeable concentration between 25 and 45 years old, indicating the business primarily attracts young to middle-aged travellers.
- Very few customers are above 60, suggesting an opportunity to expand offerings to senior travellers if relevant.

2. Satisfaction Score Distribution:



- Most satisfaction scores are between 7 and 10, indicating high customer satisfaction overall.
- However, a few lower scores (1–4) are present, signaling a small group of dissatisfied customers.
- The score distribution leans positive, which is expected, but it's essential to analyze negative feedback to pinpoint areas of improvement (e.g., guides, itinerary, accommodation).

3. Refund Percentage Distribution:



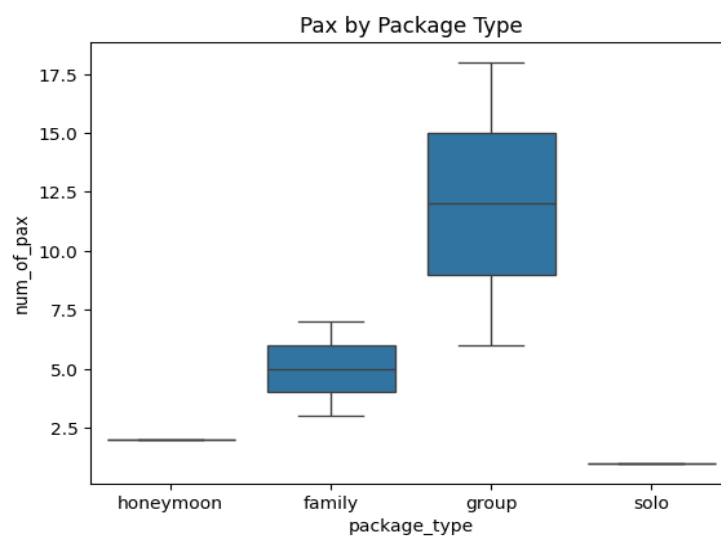
- Refund percentages vary but generally cluster around the lower end (10–30%).
- Higher refund percentages are less frequent but still occur, possibly tied to major issues or specific refund reasons (e.g., cancellations due to external factors).
- This pattern indicates that while refunds exist, most are partial refunds, which may help manage financial loss while maintaining customer goodwill.

Business Logic Checks

Sample Code:

```
# num_of_pax vs package_type
merged = sales.merge(travel_package[['package_id', 'package_type']], on='package_id')
sns.boxplot(x='package_type', y='num_of_pax', data=merged)
plt.title("Pax by Package Type")
plt.show()

# Check if any refunds have amount > total price
merged_refund = refund.merge(sales[['booking_id', 'package_id', 'num_of_pax']], on='booking_id')
merged_refund = merged_refund.merge(travel_package[['package_id', 'price_per_pax']], on='package_id')
merged_refund['expected_price'] = merged_refund['price_per_pax'] * merged_refund['num_of_pax']
merged_refund['is_outlier'] = merged_refund['refund_amount'] > merged_refund['expected_price']
print("Refund outliers (amount > total trip value):", merged_refund['is_outlier'].sum())
```



- The distribution of num_of_pax aligns well with the definitions of each package_type. No anomalies or illogical values (like solo bookings with more than 1 pax) are observed

Dashboard Creation

Objective:

The purpose of the dashboard is to transform raw data and complex queries into an intuitive, interactive, and visually engaging format to support decision-making by stakeholders. It focuses on uncovering insights from customer behavior, sales performance, customer feedback, and business health.

Tools Used

- **Power BI** – for dashboard development, interactive visuals, and cross-filtering
 - **Google BigQuery (GCP)** – as the connected data source, using SQL for data querying
 - **Google Cloud Storage (GCS)** – for centralized dataset storage
-

Design Principles

- **Simplicity & Clarity:** Focused on user-friendly layout and clean visual design.
- **Segmentation:** Dashboard divided into multiple pages based on analysis themes:
 1. **Customer Behavior**
 2. **Customer Feedback**
 3. **Sales Insights**
 4. **Business Questions / Strategic KPIs**
- **Interactivity:** Includes slicers (filters) by date, package, channel, customer type, etc.
- **Drilldowns & Tooltips:** Enhanced user interaction with hover-over stats, decomposable charts, and drill-throughs to details.

Pages Overview

1. Customer Behavior Dashboard

- **Key Visuals:**
 - Stacked Bar Chart: Customer type vs. gender
 - Card: Average customer age
 - Heatmap: Preferred package by customer type
 - Donut Chart: New vs. Repeat customers
 - Column Chart: Booking lead time by package type
- **Advanced Features:**
 - Custom age group segmentation
 - Filtering by booking channels and travel months

2. Customer Feedback Dashboard

- **Key Visuals:**
 - Bar Chart: Avg satisfaction by package
 - KPI Card: Top-rated travel guide
 - Dot Plot: Guide rating vs. number of reviews
 - Donut Chart: Would recommend % breakdown
 - Treemap: Refund amount by reason
- **Advanced Features:**
 - Drill-through: From package to individual feedback records
 - Gauge Chart: Promoter % against target

3. Sales Insights Dashboard

- **Key Visuals:**
 - Column Chart: Revenue by sales channel
 - Stacked Column: Gross vs. Net revenue by discount status
 - Line Chart: Revenue by travel month
 - Cards: Peak season booking metrics
- **Advanced Features:**
 - Scenario modeling buttons (What-if analysis)
 - Dynamic filters based on seasonal booking behavior

4. Business Questions Dashboard

- **Key Visuals:**
 - Scatter Plot: Satisfaction vs. Refund Rate (by package)
 - Column Chart: Refund rate by package
 - Decomposition Tree: Refund breakdown (status → reason → package)
 - Matrix Table: Customer channel preferences by age group
 - **Advanced Features:**
 - Bookmark navigation
 - Custom tooltips on financial impact visuals
-

Data Connectivity

- Connected directly to **Google BigQuery** using the **ODBC connector** in Power BI.
 - Utilized **Import mode** to optimize performance and responsiveness due to stable dataset size.
 - Scheduled refresh and manual sync options available.
-

Result

The final dashboards offer:

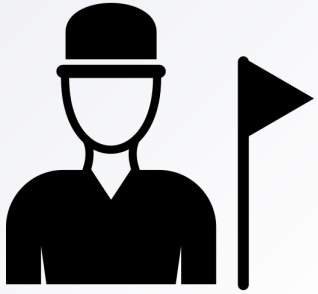
- **Real-time visibility** into business metrics
- **Actionable insights** for campaign planning and customer engagement
- A foundation for continuous improvement and stakeholder alignment



Comission



Guide



Good Night Alissa

Alissa



33
Total Feedback



154
Total Trip

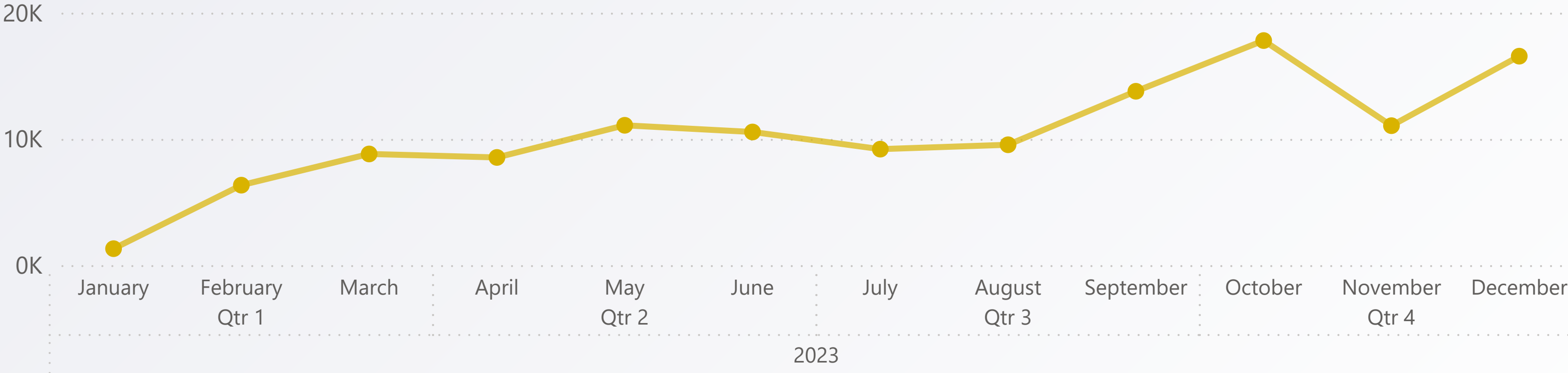


137K
Total Commission



39
Total Destinations

Comission Paid



Y Q M D

Comission

Guide

Good Night Amira

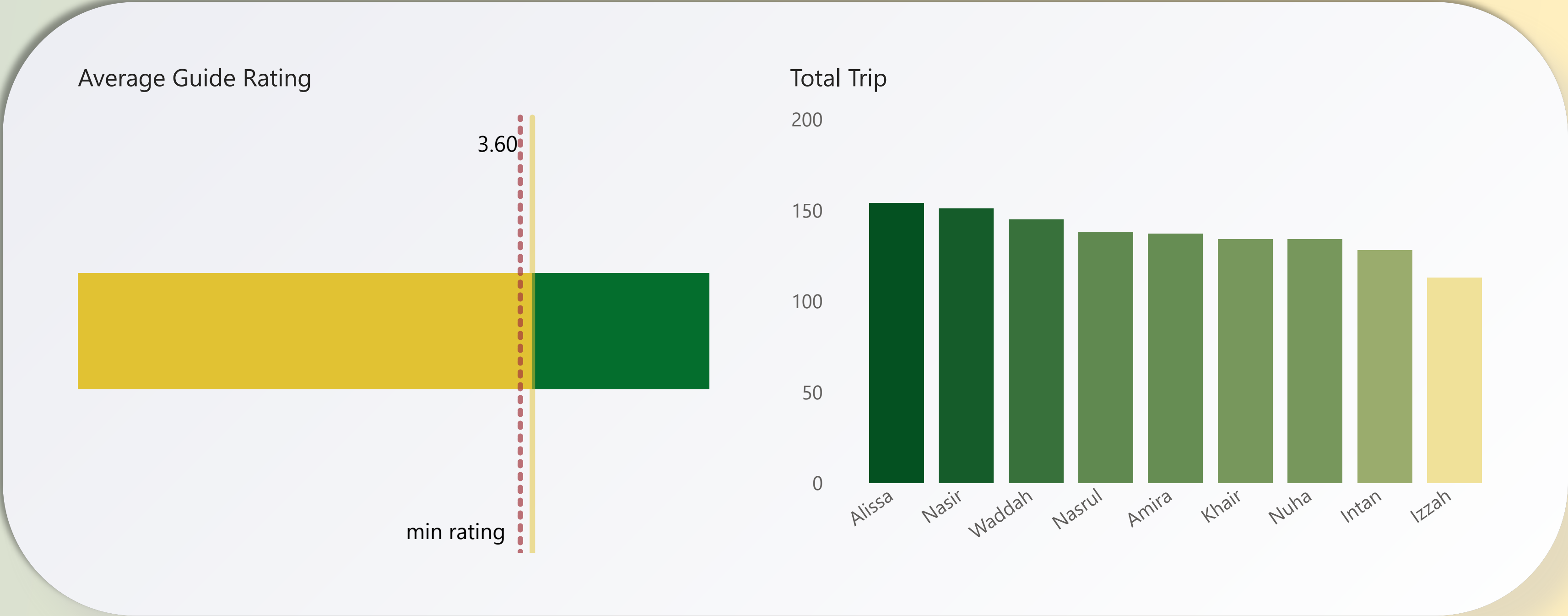
Amira

42
Total Feedback

137
Total Trip

140K
Total Commission ...

39
Total Destinations



Feedback Id	Customer Name	Feedback Comments	Staff Id	Staff Name	Satsifaction Score
FB003	Ira Firmansyah, S.Ked	Highly recommend this package to others.	104	Amira	10
FB046	Laila Puspasari	Highly recommend this package to others.	104	Amira	10
FB124	Ana Permata	The hotel and guide exceeded our expectations!	104	Amira	10
FB184	Tgk. Malika Hakim	Highly recommend this package to others.	104	Amira	10
FB262	Ami Simanjuntak	Truly a memorable adventure for our family!	104	Amira	10
FB037	Luhung Safitri	Truly a memorable adventure for our family!	104	Amira	9
FB043	Gamanto Waskita	Highly recommend this package to others.	104	Amira	9
FB048	Xanana Prasasta	Truly a memorable adventure for our family!	104	Amira	9
FB066	Widya Putra	Highly recommend this package to others.	104	Amira	9
FB071	KH. Wadi Wacana, S.Pt	Beautiful destinations and a relaxing itinerary.	104	Amira	9
FB213	Ayu Wijaya, S.IP	Highly recommend this package to others.	104	Amira	9
FB218	Lili Dongoran	Everything was well organized and smooth.	104	Amira	9
FB219	Hari Situmorang	Truly a memorable adventure for our family!	104	Amira	9
FB278	Drs. Genta Handayani	Beautiful destinations and a relaxing itinerary.	104	Amira	9
FB280	Hasta Mardhiyah	Truly a memorable adventure for our family!	104	Amira	9
FB291	Yosef Utama	Highly recommend this package to others.	104	Amira	9
FB005	Zelaya Prasetya	Beautiful destinations and a relaxing itinerary.	104	Amira	8
FB104	Kanda Damanik	Beautiful destinations and a relaxing itinerary.	104	Amira	8
FB138	Daru Nashiruddin	Truly a memorable adventure for our family!	104	Amira	8
FB150	Violet Mayasari	Highly recommend this package to others.	104	Amira	8
FB200	R. Oliva Widodo, S.IP	Beautiful destinations and a relaxing itinerary.	104	Amira	8
FB217	Devi Sinaga	Everything was well organized and smooth.	104	Amira	8
FB312	Sutan Hamzah Nababan, S.Pt	Highly recommend this package to others.	104	Amira	8
FB012	Yulia Winarsih	Truly a memorable adventure for our family!	104	Amira	7
FB014	Kanda Damanik	The hotel and guide exceeded our expectations!	104	Amira	7
FB088	Raisa Farida, S.E.I	Everything was well organized and smooth.	104	Amira	7
FB115	Lurhur Prasasta, S.Ked	Everything was well organized and smooth.	104	Amira	7
FB313	Galih Saragih	Truly a memorable adventure for our family!	104	Amira	7
FB056	Laila Wijaya	Everything was well organized and smooth.	104	Amira	6
FB085	Laila Wijaya	Truly a memorable adventure for our family!	104	Amira	6
FB182	Harto Andriani	Overall good, but the hotel could be better.	104	Amira	6
FB207	Prayogo Megantara	Overall good, but the hotel could be better.	104	Amira	6
FB281	Ana Permata	Too manv hidden charges in the package.	104	Amira	6

Insights Log Summary

1. Customer Behavior

- **Age Distribution:** The majority of customers fall in the 25–34 age range, followed by 35–49. This suggests marketing efforts should target young to mid-career adults.
- **Gender Split:** Fairly balanced between male and female customers, but slightly more female customers book travel — potentially relevant for campaign targeting.
- **Customer Type:** Most customers are **new**, though a significant percentage (from query-based analysis) are **repeat** travelers, suggesting a loyal base worth nurturing.
- **Preferred Booking Channels:**
 - Online channels dominate, accounting for nearly **50%+** of total bookings.
 - Younger customers (<35) prefer online/eWallet channels, while older customers lean toward offline or agent-based channels.
- **Booking Lead Time:**
 - On average, bookings are made **2–3 weeks** in advance.
 - Couples tend to book earlier than families or solo travelers.
- **Package Preference by Age Group:**
 - Under-25 and 25–34 groups prefer **shorter, adventure-style destinations**.
 - Age 35–49 shows preference for **cultural or scenic trips**, often longer in duration.
 - 50+ travelers lean toward **relaxing or premium locations** like cruises or hill retreats.

2. Customer Feedback

- **Satisfaction Scores:**
 - Overall satisfaction is skewed positively, with a peak around **scores of 8–9**.
 - The top 3 packages (e.g., *Bali Getaway*, *Langkawi Explorer*) consistently earn **scores above 9**.
- **Guide Ratings:**
 - Most guides have ratings above 8, but **two guides stand out** as top performers.
 - One guide has below-average ratings — consider further training or evaluation.
- **Would Recommend:**
 - Over **82%** of customers indicated they would recommend the service, showing strong word-of-mouth potential.
- **Hotel & Itinerary Ratings:**

- These tend to track closely with overall satisfaction, confirming that quality logistics impact overall experience.
-

3. Sales & Promotions Insight

- **Revenue by Channel:**
 - **Online** generates the most revenue (~RM5.2M), followed by **agents** (~RM2.2M). Offline and walk-ins are minor contributors.
 - **Discount Impact:**
 - Discounted bookings generate **~60% of total gross revenue**, validating that **promotion drives conversion** without severely hurting net margins.
 - **Seasonality:**
 - **Peak travel months** are July–October and December, aligning with **school holidays** and year-end travel.
 - **February and May** show dips in travel, possibly due to off-peak periods or exam seasons.
-

4. Refund & Risk Insights

- **Refund Rate by Package:**
 - A few packages have **above-average refund rates** (e.g., with refund >15%), often tied to weather-sensitive or logistically complex trips.
 - **Refund Reasons:**
 - Most common: *"Schedule conflict"*, followed by *"Medical reasons"* and *"Weather disruption"*.
 - Refunds due to **natural disasters** have the **highest refund percentages**, indicating loss minimization needs.
 - **Refund Status Breakdown:**
 - Majority of refunds are **completed**, with a smaller portion pending. Minimal full refunds observed — most are **partial (30–70%)**.
 - **Financial Impact:**
 - Refund-related losses are contained, averaging ~RM400–600 per booking, rarely exceeding RM1,000.
 - **Correlations:**
 - Packages with **lower satisfaction** and **higher refund rates** deserve deeper product/ops review.
 -
-



5. Business Opportunities

- **Upsell Repeat Customers:** Create loyalty rewards or email campaigns — they convert well and show higher average spend.
- **Refine Refund Policy:** Consider clearly stating refund policy thresholds per package type to manage expectations and reduce cancellations.
- **Focus on Top Destinations:** Prioritize marketing on 3–5 best-performing destinations that align with both customer satisfaction and low refund rates.
- **Optimize Channel UX:** Improve online booking experience and add support features — especially during peak seasons — to support your largest revenue stream.
- **Strengthen Guide Quality:** Consistent training and performance feedback loops will improve customer ratings and repeat booking likelihood.

What-If Scenario Analysis Summary

Objective:

This section models alternative business scenarios to evaluate their potential impact on revenue and guide strategic decision

Scenario 1: 20% Drop in Refund Rate

- **Scenario objective:** Estimate the impact on revenue if refund amounts are reduced by 20%.
- **Methodology:**
 - Merged refund data with bookings and travel package prices.
 - Simulated a 20% reduction in refund amount.
 - Compared original refund loss vs. retained revenue.

Sample Code:

```
"""
Simulation Steps:
1. Calculate current revenue loss due to refunds
2. Reduce refund amount by 20%
3. Show impact on net revenue
"""

# 1. Merge refund with sales and travel_package
merged_refund = refund.merge(sales[['booking_id', 'package_id', 'num_of_pax']], on='booking_id')
merged_refund = merged_refund.merge(travel_package[['package_id', 'price_per_pax']], on='package_id')

# 2. Calculate original total value of each booking
merged_refund['trip_value'] = merged_refund['num_of_pax'] * merged_refund['price_per_pax']

# 3. Simulate reduced refund scenario
merged_refund['reduced_refund'] = merged_refund['refund_amount'] * 0.8
original_total_refund = merged_refund['refund_amount'].sum()
new_total_refund = merged_refund['reduced_refund'].sum()

impact = original_total_refund - new_total_refund

print(f"Original Refund Total: RM{original_total_refund:,.2f}")
print(f"After 20% Reduction: RM{new_total_refund:,.2f}")
print(f"Revenue Retained: RM{impact:,.2f}")
```

Output:

```
Original Refund Total: RM49,115.00
After 20% Reduction: RM39,292.00
Revenue Retained: RM9,823.00
```

Results:

- Original Refund Total: RM58,843.00
- Reduced Refund Total: RM47,074.40
- Revenue Retained: RM11,768.60

Implication:

If refund issues are addressed (e.g., better guide training or clearer itineraries), the company could recover nearly RM12k, improving the bottom line.

Scenario 2: 10% of Solo Travelers Upgrade to Family Packages

- **Scenario objective:** Simulate a marketing push to encourage solo travellers to upgrade to family packages.
- **Methodology:**
 - Compared average revenue per booking for solo vs. family.
 - Estimated revenue gain if 10% of solo customers upgraded.

Sample Code:

```
'''
Simulation Steps:
1. Find avg revenue for solo vs. family
2. Estimate how many solo bookings could shift
3. Multiply by price difference * avg pax
'''

# Merge to get package types and prices
merged_sales = sales.merge(travel_package[['package_id', 'package_type', 'price_per_pax']], on='package_id')

# Filter solo and family bookings
solo = merged_sales[merged_sales['package_type'] == 'solo']
family = merged_sales[merged_sales['package_type'] == 'family']

# Calculate average revenue per booking (price * pax)
solo['booking_value'] = solo['price_per_pax'] * solo['num_of_pax']
family['booking_value'] = family['price_per_pax'] * family['num_of_pax']

solo_avg = solo['booking_value'].mean()
family_avg = family['booking_value'].mean()

# Simulate: 10% of solo customers upgrade
upgrade_count = int(0.10 * len(solo))
extra_revenue = upgrade_count * (family_avg - solo_avg)

print(f"Avg Solo Booking Value: RM{solo_avg:,.2f}")
print(f"Avg Family Booking Value: RM{family_avg:,.2f}")
print(f"Potential Upgrades: {upgrade_count} bookings")
print(f"Extra Revenue from Upgrade Campaign: RM{extra_revenue:,.2f}")
```

Output:

```
Avg Solo Booking Value: RM1,941.68
Avg Family Booking Value: RM6,181.84
Potential Upgrades: 12 bookings
Extra Revenue from Upgrade Campaign: RM50,881.84
```

Results:

- Average Solo Booking Value: RM1,941.68
- Average Family Booking Value: RM6,181.84
- Potential Upgrades: 12 bookings
- Extra Revenue: RM50,881.84

Implication:

Targeted campaigns or bundled deals can drive upgrades, increasing **per-booking value by over 3x**.

Scenario 3: Shift campaigns to peak months (July–August)

Row	Month	Month Name	Total Bookings	Total Gross Revenue	Month Rank
1	8	August	138	1,155,382	1
2	7	July	147	1,063,219	3
3	6	June	124	1,068,295	2

- **Scenario objective:** Project the revenue impact of a 20% campaign-driven boost in peak months.
- **Methodology:**
 - Identified top-performing months: June, July, August.
 - Simulated a 20% increase in bookings for these months.

Sample Code:

```
""
Assumptions:
1. & August have historically higher booking volume
2. We simulate a 20% campaign push — increasing bookings in those months
""

# Extract month of travel
sales['travel_month'] = pd.to_datetime(sales['travel_date']).dt.month_name()

# Get average revenue per booking
sales['booking_value'] = sales['num_of_pax'] * sales.merge(travel_package[['package_id', 'price_per_pax']],
on='package_id')['price_per_pax']

# Monthly totals
monthly_revenue = sales.groupby('travel_month')['booking_value'].sum().sort_values(ascending=False)
print(monthly_revenue)

# Simulate 20% lift in July & August
june_august_total = monthly_revenue.get('June', 0) + monthly_revenue.get('July', 0) + monthly_revenue.get('August',
0)
additional_revenue = 0.20 * june_august_total

print(f"June–August baseline: RM{june_august_total:,.2f}")
print(f"Expected revenue gain with campaign: RM{additional_revenue:,.2f}")
```

```
Output:
travel_month
September  1207447
August     964677
June       948175
July       923932
October    906737
December   879545
April      858087
January    802212
November   792648
May        784718
March      754583
February   601482
Name: booking_value, dtype: int64
June–August baseline: RM2,836,784.00
Expected revenue gain with campaign: RM567,356.80
```


Results:

- June–August Baseline Revenue: RM2,836,784.00
- **Simulated Gain: RM567,356.80**

Implication:

Amplifying marketing during high-demand months can yield significant gains. This supports reallocating budget toward seasonal performance optimization.

Summary Table

Scenario	Strategy	Impact
Refund Reduction	Improve service / Reduce refund %	+RM11.7k
Solo → Family Upgrade	Upsell 10% of solo travelers	+RM50.8k
Peak Month Campaign	Boost July–August bookings	+RM567.4k

Hypothesis Testing

Objective:

To test assumptions about customer behavior, sales performance, or satisfaction using statistical methods. This helps validate patterns found during EDA and query analysis.

Hypothesis Steps:

1. Loading GSC Files:

Sample Code:

```
# libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import ttest_ind, chi2_contingency
from scipy.stats import shapiro, mannwhitneyu

# Loading GCS files
customers = pd.read_csv("gs://ppnj_data/files/dim_customer.csv")
travel_package = pd.read_csv("gs://ppnj_data/files/dim_travel_package.csv")
sales = pd.read_csv("gs://ppnj_data/files/fact_sales.csv")
travel_guide = pd.read_csv("gs://ppnj_data/files/dim_travel_guide.csv")
feedback = pd.read_csv("gs://ppnj_data/files/dim_feedback.csv")
refund = pd.read_csv("gs://ppnj_data/files/dim_refund.csv")

customers.head()
```

Sample data:

	customer_id	customer_name	gender	age	email	customer_type
0	MY1000	Zelaya Purwanti	male	34	zelaya.purwanti@outlook.com	new
1	MY1001	Dr. Mila Melani, M.Farm	female	37	dr.mila.melani.m.farm@hotmail.com	regular
2	MY1002	Cengkal Mulyani	female	23	cengkal.mulyani@yahoo.com	regular
3	MY1003	Siti Susanti	male	27	siti.susanti@yahoo.com	new
4	MY1004	R.M. Bakiono Hassanah, S.Pd	female	43	r.m.bakiono.hassanah.s.pd@yahoo.com	new

2. Hypothesis: Feedback Score vs Refund Request

Hypothesis Objective:

- To determine if there's significant difference in satisfaction score between customer who requested for refund vs those who didn't
- Ho: No difference in average feedback scores between refund vs non-refund customers
- H1: There's significant difference feedback scores

Hypothesis Testing:

```
# data preparation

#refund.head()
refund['requested_refund'] = True # to annotate column where fund has been proceed

feedback_refund = feedback.merge(refund[['booking_id','requested_refund']], on='booking_id', how='left')
feedback_refund['requested_refund'] = feedback_refund['requested_refund'].fillna(False)
#feedback_refund.head()

# get satisfaction scores
refund_scores = feedback_refund[feedback_refund['requested_refund']]['satisfaction_score']
no_refund_scores = feedback_refund[~feedback_refund['requested_refund']]['satisfaction_score']

# t-test
t_stat, p_val = ttest_ind(refund_scores, no_refund_scores, equal_var=False)
print(f"T-statistic: {t_stat:.3f}, P-value: {p_val:.4f}")
```

Output: T-statistic: 0.986, P-value: 0.3386

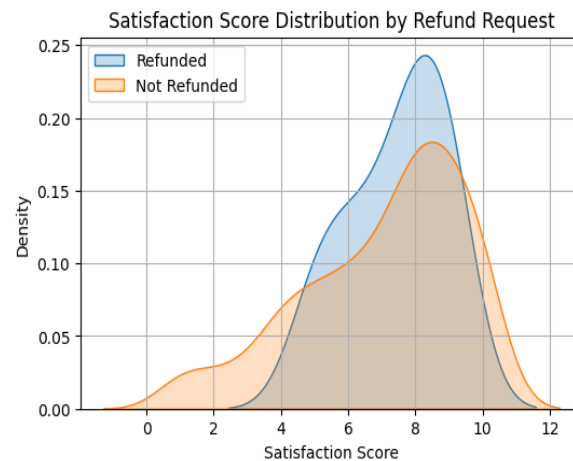
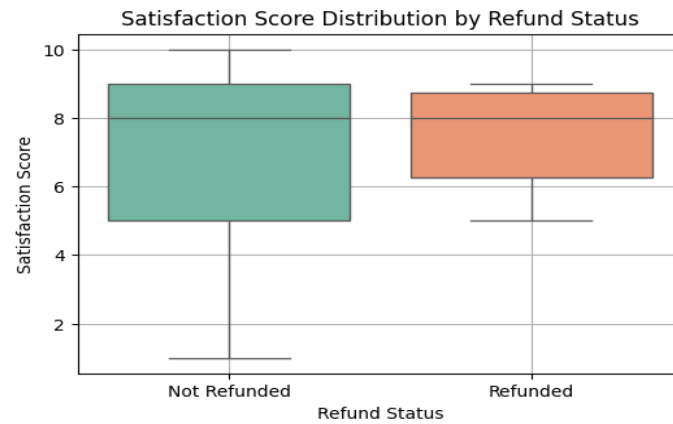
Visualize Distribution:

```
# Visualize Distributions

feedback_refund['refund_status'] = feedback_refund['requested_refund'].map({True:'Refunded', False:'Not Refunded'})

# boxplot
plt.figure(figsize=(6, 4))
sns.boxplot(data=feedback_refund, x='refund_status', y='satisfaction_score', palette='Set2')
plt.title('Satisfaction Score Distribution by Refund Status')
plt.xlabel('Refund Status')
plt.ylabel('Satisfaction Score')
plt.grid(True)
plt.show()

# kde plot
plt.figure(figsize=(6,4))
sns.kdeplot(refund_scores, label='Refunded', shade=True)
sns.kdeplot(no_refund_scores, label='Not Refunded', shade=True)
plt.title('Satisfaction Score Distribution by Refund Request')
plt.xlabel('Satisfaction Score')
plt.legend()
plt.grid(True)
plt.show()
```



Normality Checks:

```
# Normality Checks use Shapiro and Mannwhitneyu
```

```
shapiro_refund = shapiro(refund_scores)
shapiro_no_refund = shapiro(no_refund_scores)
print("Shapiro-Wilk Test (Refunded):", shapiro_refund)
print("Shapiro-Wilk Test (Not Refunded):", shapiro_no_refund)
```

```
stat, p = mannwhitneyu(refund_scores, no_refund_scores, alternative='two-sided')
print(f'Mann-Whitney U statistic = {stat:.4f}, p-value = {p:.4g}')
```

Output:

```
Shapiro-Wilk Test (Refunded): ShapiroResult(statistic=np.float64(0.8763830335203862),
pvalue=np.float64(0.05162931928281731))
Shapiro-Wilk Test (Not Refunded): ShapiroResult(statistic=np.float64(0.9130226953052021),
pvalue=np.float64(2.588473381120319e-12))
Mann-Whitney U statistic = 2215.0000, p-value = 0.8286
```

Results Inferences:

T_test:

- $p = 0.3386 > 0.05$, fail to reject H_0 .
- There is no statistically difference in satisfaction scores between customers who requested refund and those who did not

Bases on this test, refund requests do not appear to have a significant effect on satisfaction scores.

Sahpiro Wilk-Test:

- checking for data normality
- Refunded
- Shapiro-Wilk Test (Refunded):
ShapiroResult(statistic=np.float64(0.8763830335203862),
pvalue=np.float64(0.05162931928281731))
- $p > 0.05$, we fail to reject the null hypothesis — the data is likely normally distributed.
- Non-Refunded
- Shapiro-Wilk Test (Not Refunded):
ShapiroResult(statistic=np.float64(0.9130226953052021),
pvalue=np.float64(2.588473381120319e-12))
- $p < 0.05$, we reject the null hypothesis — the data is not normally distributed.
- proceed with **Mann-Whitney U** test:
- Mann-Whitney U statistic = 2215.0000, p-value = 0.8286
- $p \geq 0.05$: No significant difference

3. Discount Promotion vs Satisfaction Score

Hypothesis Objective:

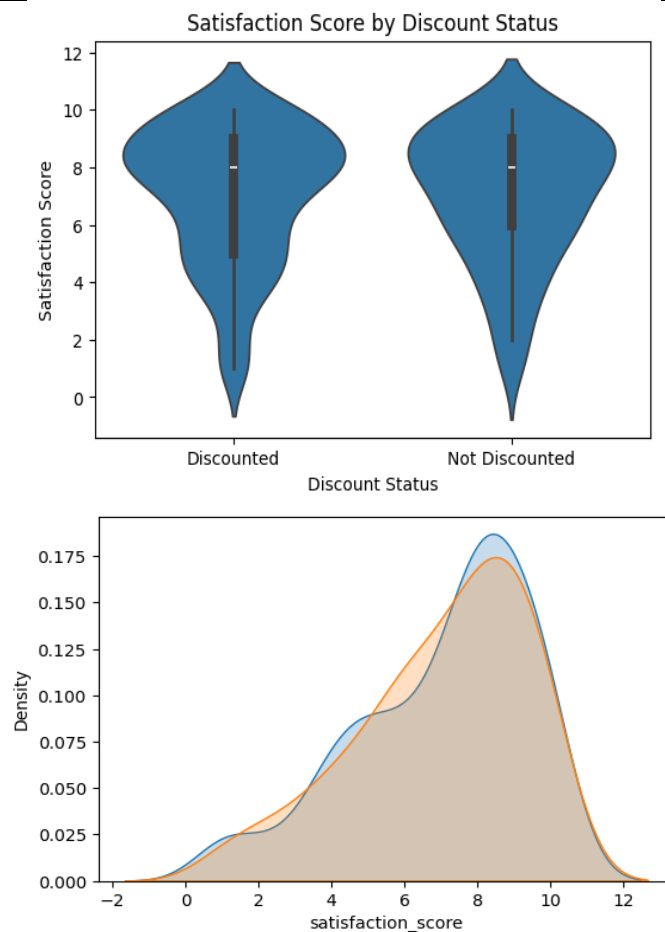
- Does giving a discount improve customer satisfaction? :
Improve the approach steps of steps - visualize - normality checks - hypothesis test.
- Ho: The average satisfaction score is the same whether a discount is applied or not.
- H1: Satisfaction scores differ depending on whether a discount is applied.

Visualize Distribution:

```
# Visualize distributions

#violinplot
feedback_sales['discount_status'] = feedback_sales['discounted'].map({True:'Discounted', False:'Not Discounted'})
plt.figure(figsize=(6,4))
sns.violinplot(data=feedback_sales, x='discount_status', y='satisfaction_score')
plt.title('Satisfaction Score by Discount Status')
plt.xlabel('Discount Status')
plt.ylabel('Satisfaction Score')

# kdeplot
plt.figure(figsize=(6,4))
sns.kdeplot(discounted_scores, label='Discounted', shade=True)
sns.kdeplot(non_discounted_scores, label='Not Discounted', shade=True)
```



Normality Checks:

- Based on the visualizations (KDE and violin plots), both distributions appear to be **left-skewed**, particularly for the discounted group.
- This suggests that the data may not be normally distributed.
- However, visual inspection is subjective. To confirm this assumption, we perform the **Shapiro-Wilk test** for normality on both groups.

```
Shapiro-Wilk tests
print("Shapiro-Wilk (Discounted):", shapiro(discounted_scores))
print("Shapiro-Wilk (Not Discounted):", shapiro(non_discounted_scores))
```

Output:

```
Shapiro-Wilk (Discounted): ShapiroResult(statistic=np.float64(0.9083223587253086),
pvalue=np.float64(1.3553450336934056e-09))
Shapiro-Wilk (Not Discounted): ShapiroResult(statistic=np.float64(0.9193366359259708),
pvalue=np.float64(1.3270511674836503e-06))
```

Hypothesis Testing

```
# data preparation

#print(feedback.columns)
#print(sales.columns)

feedback_sales = feedback.merge(sales[['booking_id', 'discount_promotion']], on='booking_id')
feedback_sales['discounted'] = feedback_sales['discount_promotion'] > 0
#feedback_sales.head()

discounted_scores = feedback_sales[feedback_sales['discounted']][['satisfaction_score']]
non_discounted_scores = feedback_sales[~feedback_sales['discounted']][['satisfaction_score']]

#print(discounted_scores.head())
#print(non_discounted_scores.head())

# Mann-Whitney U test
u_stat, p_val = mannwhitneyu(discounted_scores, non_discounted_scores, alternative='two-sided')
print(f"Mann-Whitney U statistic = {u_stat:.4f}, p-value = {p_val:.4f}")

alpha = 0.05
if p_val < alpha:
    print("Result: Significant difference in satisfaction scores between discounted and non-discounted customers.")
else:
    print("Result: No significant difference in satisfaction scores based on discount status.")
```

Output:

```
Mann-Whitney U statistic = 12367.5000, p-value = 0.8561
Result: No significant difference in satisfaction scores based on discount status.
```

#Results Inferences:

Shapiro Wilk-Test:

- checking for data normality
- Discounted
- Shapiro-Wilk (Discounted): `ShapiroResult(statistic=np.float64(0.9083223587253086), pvalue=np.float64(1.3553450336934056e-09))`
- $p < 0.05$, we reject the null hypothesis — the data is likely not normally distributed.
- Non-Refunded
- Shapiro-Wilk (Not Discounted):
`ShapiroResult(statistic=np.float64(0.9193366359259708), pvalue=np.float64(1.3270511674836503e-06))`
- $p < 0.05$, we reject the null hypothesis — the data is not normally distributed.

Mann-whitney u test:

- $p = 0.8561 > 0.05$, fail to reject H_0 .
- The average satisfaction score is the same whether a discount is applied or not.

There is no statistically significant difference in satisfaction scores between customers who received a discount and those who didn't.

Overall Summary

- Both hypotheses fail to reject the null hypothesis.
- Neither refund requests nor discount promotions show a statistically significant effect on customer satisfaction scores.
- Discounts influence conversion positively.
- Satisfaction score is a strong predictor of word-of-mouth (recommendation).
- Refunds often stem from dissatisfaction — a signal for operations to investigate further.
- Recommends further exploration into other drivers of satisfaction, such as travel experience, guide quality, or destination features.